

Design a Parking Lot

Amazon · Microsoft · Goldman Sachs · Uber · Salesforce
· Adobe · Walmart · Flipkart · Swiggy · Oracle · PayPal –
ఈ ఒక్క question అన్ని చోట్లా వస్తుంది.

ఇది సులభంగా కనిపిస్తుంది, అందుకే ప్రమాదకరం. ఇక్కడ మనం ఐదు lines తో మొదలుపెట్టి, మన design ని **మూడుసార్లు విరగొట్టి**, ప్రతిసారీ ఎందుకు విరిగిందో చూసి, ఆపై సరిచేస్తాం. ప్రతి output నిజంగా **node** lo run చేసినదే.

ఈ doc ఎలా చదవాలి

పక్కన ఒక editor తెరిచి, కలిసి రాయండి. ప్రతి దశలో ఒక చిన్న code ముక్క ఉంటుంది – దాన్ని type చేసి `node` lo run చేసి, ఇక్కడ చూపించిన output వస్తుందే చూడండి.

PARKING LOT ఎందుకు ఇంత ముఖ్యం

ఇది "అత్యధికంగా అడిగే LLD question" అని చాలా చోట్ల చెప్తారు, మరియు దానికి ఒక మంచి కారణం ఉంది: ఒక చిన్న, అందరికీ తెలిసిన domain lo **inheritance, composition, concurrency, మరియు మారుతున్న business rules** – నాలుగూ ఒకేసారి ఉన్నాయి.

అంటే interviewer ఒక్క ప్రశ్నతో మీ గురించి నాలుగు విషయాలు తెలుసుకోగలడు. అందుకే ఇది మొదటి question, మరియు అందుకే దీన్ని పూర్తిగా master చేయాలి.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Interviewer అడిగింది ఏమిటి – మరియు ఇది ఎందుకు మోసపుచ్చుతుంది
2. మనసులో ఒకసారి car park చేసి చూద్దాం
3. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి version, మరియు మొదటి విరుపు

4. Step – అతి సులభమైన lot
5. మొదటి విరుపు – bike spot lo ఒక truck
6. Step – Spot కి ఒక size ఇవ్వడం

Part 3 – రెండో విరుపు: రెండు gates, ఒకే spot

7. రెండో విరుపు – double booking ని కళ్ళతో చూడటం
8. Step – వెతకడం + ఇవ్వడం ఒకే ఆపరేషన్ గా
9. నిజమైన systems lo ఇది ఎలా ఉంటుంది

Part 4 – మూడో విరుపు: పెరుగుతున్న pricing

10. మూడో విరుపు – ఒక్కో నియమానికి ఒక్కో `if`
11. Step – Pricing ని బయట పెట్టడం
12. Step – నియమాలని ఒకదాని మీద ఒకటి పేర్చడం (Decorator)

Part 5 – Spot ఎంపిక, floors

13. Step – "ఏ spot ఇవ్వాలి" కూడా ఒక మారే నియమమే
14. Floors, మరియు `Floor` ని class చేయాలా వద్దా

Part 6 – పూర్తి system

15. మొత్తం code ఒకే చోట
16. నడిపి చూద్దాం – ticket నుంచి బిల్లు వరకు

Part 7 – Interview lo

17. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
18. నోటితో చెప్పాల్సిన English script
19. Follow-ups – మరియు మన design ఎలా తట్టుకుంటుంది
20. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

ఈ Part lo ఒక్క పంక్తి code కూడా లేదు. అది ఉద్దేశపూర్వకం – interview lo కూడా మొదటి పది నిమిషాలు ఇలాగే ఉండాలి.

1. Interviewer అడిగింది ఏమిటి – మరియు ఇది ఎందుకు మోసపుచ్చుతుంది

"Design a parking lot."

మూడు పదాలు. అందరికీ తెలిసిన విషయం. అందుకే ఇది ప్రమాదకరం – చాలా మంది వెంటనే code రాయడం మొదలుపెడతారు, మరియు అక్కడే ఓడిపోతారు.

ఒక్క క్షణం ఆగి ఆలోచిస్తే, ఈ ఒక్క వాక్యంలో నాలుగు వేర్వేరు సమస్యలు దాగి ఉన్నాయి:

"Design a parking lot" – లోపల ఏమి ఉంది

1 - రకాలు (Modelling)

Bike, car, truck – ఒక్కదానికి ఒక్కే చోటు
ఇది inheritance నా composition నా?

కొలుస్తుంది: OOP వునాది

2 - పోటీ (Concurrency)

రెండు gates, ఒకే చివరి spot
ఇద్దరికీ ఇచ్చేస్తే?
కొలుస్తుంది: seniority – ఇదే అసలు filter

3 - మారే నియమాలు (Pricing)

గంటకి ఇంత • weekend lo ఎక్కువ
మొదటి 30 నిమిషాలు ఉచితం • EV discount

కొలుస్తుంది: Open/Closed

4 - ఎంపిక (Allocation)

ఏ spot ఇవ్వాలి – దగ్గరిదా, సరిపోయేదా?
ఇది కూడా మారే నియమమే

కొలుస్తుంది: abstraction సరైన చోటు ఉందా

ఇక్కడ చాలా మంది ఓడతారు

సమస్య 1 ని అందరూ చేస్తారు. సమస్య 2 ని **మరే లేవనెత్తాలి** – interviewer అడిగేవరకు ఆగితే ఆలస్యం.
సమస్యలు 3, 4 ని ఒక **if-else** తో పరిష్కరిస్తే – అది పనిచేస్తుంది, కానీ మిమ్మల్ని junior గా చూపిస్తుంది.

ఈ doc lo మనం ఈ నాలుగింటినీ **ఒక్కొక్కటిగా** ఎదుర్కొంటాం – మరియు ప్రతిసారీ, ముందు తప్పు పరిష్కారం రాసి, అది విరిగే చోటు చూసి, ఆపై సరిచేస్తాం.

2. మనసులో ఒకసారి car park చేసి చూద్దాం

Code కి ముందు ఒక నిజమైన దృశ్యం. మీరు మీ car తో ఒక mall parking lo ప్రవేశిస్తున్నారు.

1. **Gate దగ్గర ఆగారు.** ఒక machine మీ number plate చదివి, ఒక **ticket** ఇచ్చింది.
2. Ticket మీద ఏముంది? **ఏ spot, ఎప్పుడు వచ్చారు**, మరియు ఒక ID.
3. మీరు ఆ spot కి వెళ్ళి car ఆపారు.
4. తిరిగి వచ్చేటప్పుడు ticket చూపించారు. Machine **ఎంతసేపు ఉన్నారో లెక్కించి** డబ్బు అడిగింది.

ఇప్పుడు ఆలోచించండి – ఈ నాలుగు అడుగుల్లో **system ఏమి గుర్తుపెట్టుకోవాలి?**

- ఏ spots ఉన్నాయి, ఏవి ఖాళీ – ఇది **lot** యొక్క జ్ఞానం.
- ఈ car ఎక్కడ ఉంది, ఎప్పుడు వచ్చింది – ఇది **ticket** యొక్క జ్ఞానం.
- ఎంత డబ్బు – ఇది ఒక **నియమం**, ఒక వస్తువు కాదు.

ఆ చివరి పంక్తిని గమనించండి. "ఎంత డబ్బు" అనేది ఒక **data** కాదు – అది ఒక **లెక్కింపు నియమం**. మరియు నియమాలు **మారతాయి**. ఈ ఒక్క గుర్తింపు §11 lo మన design ని మిగతా అందరి design నుంచి వేరు చేస్తుంది.

ఒక ముఖ్యమైన ప్రశ్న: ticket ఎందుకు కావాలి?

"Car ఎక్కడ ఉందో spot lo store చేస్తే చాలు కదా?" – అవును, కానీ అప్పుడు **ఎప్పుడు వచ్చిందో** ఎక్కడ పెడతాం? Spot lo పెడితే, ఆ car వెళ్ళిపోయాక ఆ సమాచారం పోతుంది, మరియు bill చరిత్ర ఉండదు.

Ticket = ఒక car మరియు ఒక spot మధ్య ఒక నిర్దిష్ట కాలపు సంబంధం. అది ఒక స్వతంత్ర వస్తువు కావాలి. ఇది interview lo చెప్పదగిన మంచి పరిశీలన.

3. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్నలు అడగడం ఒక ఆచారం కాదు – ప్రతి జవాబూ నా design ని మార్చాలి. లేకపోతే ఆ ప్రశ్న అడగడం వృధా.

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఎన్ని రకాల vehicles?	ఇదే Spot కి size కావాలా వద్దా అని తేలుస్తుంది (§6)
ఒక rakam vehicle పెద్ద spot lo పార్క్ చేయొచ్చా?	"అవును" అంటే <code>size >= vehicle.size</code> ; "కాదు" అంటే కచ్చితమైన match
ఎన్ని gates?	ఇదే అత్యంత ముఖ్యమైన ప్రశ్న – ఒకటి కంటే ఎక్కువ అంటే concurrency (§7)
Pricing ఎలా?	గంటకి ఒకటే rate నా, లేక రకం/సమయం బట్టి మారుతుందా (§10)
ఏ spot ఇవ్వాలి – ఏదైనా, లేక ఒక నియమం ఉందా?	"ఏదైనా" అంటే simple; నియమం ఉంటే Strategy (§13)
Floors ఉన్నాయా?	ఉంటే spot ID lo floor, మరియు "దగ్గరి spot" అనే భావన
Payment ఇక్కడే జరుగుతుందా?	Scope ని కుదించడానికి – నేను దీన్ని బయట పెడతాను

"ఎన్ని GATES?" – ఈ ఒక్క ప్రశ్న మీ LEVEL ని చూపిస్తుంది

చాలా మంది candidates ఈ ప్రశ్న అడగరు, మరియు ఒకే gate ఉన్నట్టు design చేస్తారు. తర్వాత interviewer అడుగుతాడు: "ఇప్పుడు నాలుగు gates ఉన్నాయి, రెండు cars ఒకేసారి వస్తే?" – అప్పుడు వాళ్ళ design మొత్తం తిరిగి రాయాలి.

మీరు మొదట్లోనే ఈ ప్రశ్న అడిగితే, రెండు విషయాలు జరుగుతాయి: మీ design మొదటి నుంచే సరైన ఆకారంలో ఉంటుంది, మరియు interviewer కి "ఇతను concurrency గురించి ఆలోచిస్తున్నాడు" అని వెంటనే తెలుస్తుంది.

Scope ని మీరే కుదించండి

45 నిమిషాల్లో అన్నీ చేయలేరు. కాబట్టి మీరే కొన్నిటిని బయట పెట్టండి – ఇది బలహీనత కాదు, maturity:

"Payment gateway integration, number-plate recognition, మరియు reservation – ఈ మూడింటినీ నేను scope బయట పెడుతున్నాను. నా దృష్టి spot allocation, ticketing, మరియు pricing మీద ఉంటుంది. సమయం మిగిలితే వాటికి తిరిగి వస్తాను."

Part 2 – మొదటి version, మరియు మొదటి విరుపు

అతి సులభమైనదాంతో మొదలుపెడదాం. అది విరిగేవరకు.

4. Step – అతి సులభమైన lot

అతి సులభమైన lot ఏమిటి? **N** చోట్లు, ఒక car వస్తే ఖాళీ చోటు ఇవ్వడం, వెళ్ళిపోతే ఖాళీ చేయడం. అంతే.

```
class ParkingLot {
  constructor(n) {
    this.spots = Array(n).fill(null);    // null = ఖాళీ
  }

  park(vehicle) {
    const i = this.spots.findIndex((s) => s === null);
    if (i === -1) return null;           // చోటు లేదు
    this.spots[i] = vehicle;
    return { spot: i, vehicle };         // ఇదే ticket
  }

  unpark(ticket) {
    this.spots[ticket.spot] = null;
  }
}
```

ఇది ఏం చేస్తుంది: మొదటి ఖాళీ చోటుని వెతికి, అందులో vehicle ని పెట్టి, ఒక ticket తిరిగి ఇస్తుంది.

నడిపి చూద్దాం:

```
const lot = new ParkingLot(3);
console.log('bike →', lot.park({ type: 'bike' }));
console.log('car →', lot.park({ type: 'car' }));
console.log('truck →', lot.park({ type: 'truck' }));
console.log('ఇంకొక bike →', lot.park({ type: 'bike' }));
```

```
bike → { spot: 0, vehicle: { type: 'bike' } }
car → { spot: 1, vehicle: { type: 'car' } }
truck → { spot: 2, vehicle: { type: 'truck' } }
ఇంకొక bike → null
```

పనిచేస్తోంది. మూడు vehicles పార్క్ అయ్యాయి, నాలుగోదానికి చోటు లేదని `null` వచ్చింది. **ఇది సరైనదే అనిపిస్తుంది.**

5. మొదటి విరుపు – bike spot lo ఒక truck

ఇప్పుడు ఒక నిజమైన పరిస్థితి పెడదాం. ఒక lot lo **మూడు motorcycle spots మాత్రమే** ఉన్నాయి. అక్కడికి ఒక truck వచ్చింది.

```
const bikeLot = new ParkingLot(3); // మూడు motorcycle spots
const t = bikeLot.park({ type: 'truck' });
console.log('Truck ticket:', t);
```

```
Truck ticket: { spot: 0, vehicle: { type: 'truck' } }
=> motorcycle spot lo ఒక truck. మన code దీన్ని ఆపలేదు.
```

మొదటి విరుపు

మన code ఆ truck ని **అనందంగా** ఒక motorcycle spot lo పార్క్ చేసింది. ఒక ticket కూడా ఇచ్చింది. భౌతికంగా ఇది అసాధ్యం – కానీ మన model కి ఆ విషయం తెలియదు.

తప్పు ఎక్కడ? – "అన్ని చోట్లా ఒకేలా ఉంటాయి" అనే ఊహ. మన spots అనేది కేవలం null లేదా vehicle ఉన్న ఒక array. ఒక చోటుకి తన సొంత లక్షణం లేదు.

అంటే Spot ఒక class కావాలి, ఒక array slot కాదు.

ఇక్కడ ఒక design నిర్ణయం: inheritance నా, ఒక field నా?

Vehicle రకాలని ఎలా model చేయాలి? రెండు దారులు:

దారి 1 – Inheritance. class Bike extends Vehicle, class Car extends Vehicle, class Truck extends Vehicle.

దారి 2 – ఒక size field. new Vehicle('TS-01', Size.SMALL).

ఏది సరైనది? ప్రశ్న ఇది: **ఆ రకాలు వేర్వేరుగా ప్రవర్తిస్తాయా?**

ఇప్పటివరకు లేదు – bike, car, truck మధ్య తేడా కేవలం **ఎంత చోటు కావాలి** అనేదే. ప్రవర్తనలో తేడా లేదు. కాబట్టి మూడు classes రాయడం అంటే **ఖాళీ classes** సృష్టించడం.

Interview lo ఇలా చెప్పండి: "నేను ఇప్పుడు inheritance వాడట్లేదు, ఎందుకంటే bike, car, truck మధ్య ప్రవర్తనలో తేడా లేదు – కేవలం size తేడా. ఒక enum సరిపోతుంది. కానీ ఒకవేళ truck కి ప్రత్యేక నియమాలు వస్తే (ఉదా: hazmat trucks కి ప్రత్యేక floor), అప్పుడు నేను subclass చేస్తాను. **తేడా data lo ఉంటే field; తేడా ప్రవర్తనలో ఉంటే class.**"

ఆ చివరి వాక్యం ఒక నియమంగా గుర్తుపెట్టుకోండి – ఇది ప్రతి LLD problem lo పనికొస్తుంది.

6. Step – Spot కి ఒక size ఇవ్వడం

ఇప్పుడు Spot ని ఒక నిజమైన class చేద్దాం, దానికి ఒక size ఇద్దాం:

```
const Size = Object.freeze({ SMALL: 1, MEDIUM: 2, LARGE: 3 });

class Spot {
  constructor(id, size) {
    this.id = id;
    this.size = size;
    this.vehicle = null;
  }
  get isFree() { return this.vehicle === null; }

  // ఖాళీగా ఉండాలి, మరియు ఈ vehicle పట్టంత పెద్దదై ఉండాలి
  canFit(v) { return this.isFree && this.size >= v.size; }
}
```

Size విలువలు 1, 2, 3 ఎందుకంటే – అప్పుడు "పట్టుతుందా?" అనేది ఒక్క పోలిక అవుతుంది:

this.size >= v.size. Strings వాడితే ఇక్కడ ఒక mapping table రాయాల్సి వచ్చేది.

ఇది "ఒక చిన్న vehicle పెద్ద spot lo పార్క్ చేయొచ్చు" అని చెప్తోంది – §3 lo మనం అడిగిన ప్రశ్నకి జవాబు.

Interviewer "కాదు, కచ్చితమైన match కావాలి" అంటే ఇది === అవుతుంది. **ఒక్క అక్షరం మార్పు** – ఎందుకంటే ఆ నియమం ఒకే చోట ఉంది.

ఇప్పుడు lot:

```
class ParkingLot {
  constructor(spots) { this.spots = spots; }

  findFreeSpot(v) { return this.spots.find((s) => s.canFit(v)) ?? null; }
  assign(spot, v) { spot.vehicle = v; return { spotId: spot.id, vehicle: v }; }

  park(v) {
    const spot = this.findFreeSpot(v); // ← అడుగు 1: వెతకడం
    if (!spot) return null;
    return this.assign(spot, v); // ← అడుగు 2: ఇవ్వడం
  }
}
```

నడిపి చూద్దాం:

```
bike → { spotId: 'S1', vehicle: { no: 'TS-01', size: 1 } }
car → { spotId: 'M1', vehicle: { no: 'TS-02', size: 2 } }
truck → { spotId: 'L1', vehicle: { no: 'TS-03', size: 3 } }
bike-only lot lo truck → null
```

చివరి పంక్తి చూడండి – **ఇప్పుడు truck ని సరిగ్గా తిరస్కరించింది**. మొదటి విరుపు సరిచేయబడింది.

కానీ ఆ `park()` method ని మరోసారి చూడండి. అందులో **రెండు వేర్వేరు అడుగులు** ఉన్నాయి – వెతకడం, ఆపై ఇవ్వడం. ఇప్పుడు అది హానికరంగా కనిపించట్లేదు. తర్వాతి Part lo అదే మన design ని రెండోసారి విరగ్గొడుతుంది.

Part 3 – రెండో విరుపు: రెండు gates, ఒకే spot

ఇదే ఈ problem యొక్క నిజమైన కేంద్రం, మరియు ఇక్కడే interview గెలుస్తారు లేదా ఓడతారు.

7. రెండో విరుపు – double booking ని కళ్ళతో చూడటం

§3 lo మనం అడిగాం: "ఎన్ని gates?" జవాబు "నాలుగు" అనుకుందాం.

ఇప్పుడు ఒక దృశ్యం. Lot lo **చివరి ఒక్క spot** మిగిలి ఉంది. Gate A దగ్గర ఒక car, Gate B దగ్గర ఇంకొక car – **ఒకే క్షణంలో**.

JavaScript single-threaded కాబట్టి ఇక్కడ నిజమైన race రాదు. కానీ Java lo రెండు threads చేసే పనిని మనం **చేతితో అదే క్రమంలో** నడపొచ్చు:

```
const lot = new ParkingLot([new Spot('M1', Size.MEDIUM)]); // చివరి ఒక్క spot
const carA = { no: 'AP-09', size: Size.MEDIUM };
const carB = { no: 'KA-05', size: Size.MEDIUM };

const spotForA = lot.findFreeSpot(carA); // Gate A: "M1 ఖాళీ"
const spotForB = lot.findFreeSpot(carB); // Gate B: "M1 ఖాళీ" ← A ఇంకా assign చేయలేదు
const ticketA = lot.assign(spotForA, carA);
const ticketB = lot.assign(spotForB, carB);

console.log('Gate A ticket:', ticketA.spotId, ticketA.vehicle.no);
console.log('Gate B ticket:', ticketB.spotId, ticketB.vehicle.no);
console.log('M1 lo నిజంగా ఉన్న car:', lot.spots[0].vehicle.no);
```

```
Gate A ticket: M1 AP-09
Gate B ticket: M1 KA-05
M1 lo నిజంగా ఉన్న car: KA-05
=> ఇద్దరికీ M1 ఇచ్చాం. ఒకరు వచ్చి చూస్తే చీటు లేదు.
```

రెండో విరుపు – ఇదే INTERVIEW యొక్క అసలు ప్రశ్న

ఇద్దరికీ M1 ఇచ్చేశాం. ఇద్దరూ చెల్లుబాటు అయ్యే tickets తీసుకుని లోపలికి వెళ్తారు. AP-09 వాడు M1 కి వెళ్తే అక్కడ KA-05 ఉంటుంది.

ఇది ఒక అరుదైన సమస్య కాదు. నాలుగు gates ఉన్న ఒక mall lo, పండుగ రోజున, చివరి spots కోసం ఇది **ప్రతిరోజూ** జరుగుతుంది.

తప్పు ఎక్కడ ఉంది?

park() lo రెండు అడుగులు ఉన్నాయి:

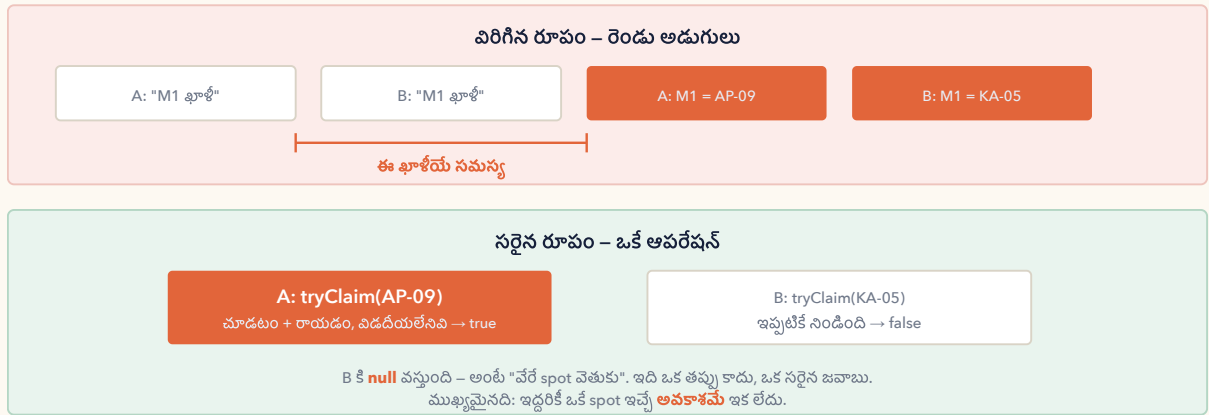
```
const spot = this.findFreeSpot(v); // అడుగు 1 - చూడటం
this.assign(spot, v);             // అడుగు 2 - రాయడం
```

ఈ రెండింటి మధ్యలో ఒక ఖాళీ ఉంది. ఆ ఖాళీలో ఇంకొక gate దూరి, అదే spot ని చూసి, అదే నిర్ణయం తీసుకోగలదు.

దీనికి ఒక పేరు ఉంది: **check-then-act race condition**. Interview lo ఈ పేరు వాడండి - ఇది మీరు దీన్ని ఒక సాధారణ నమూనాగా గుర్తిస్తున్నారని చూపిస్తుంది.

CHECK-THEN-ACT • ఖాళీ ఎక్కడ ఉంది

రెండు gates, ఒకే spot - సమయం ఎడమ నుంచి కుడికి



8. Step - వెతకడం + ఇవ్వడం ఒకే ఆపరేషన్ గా

పరిష్కారం: ఆ రెండు అడుగులని **విడదీయలేని ఒక్క ఆపరేషన్** గా మార్చడం. మరియు ఆ ఆపరేషన్ ఎక్కడ ఉండాలి? - **spot మీదే**, ఎందుకంటే ఆ spot యొక్క స్థితిని తెలిసినది, కాపాడాల్సినది అదే.

```
class Spot {
  #vehicle = null; // బయటివాళ్ళు నేరుగా మార్చకూడదు

  constructor(id, size) { this.id = id; this.size = size; }
  get isFree() { return this.#vehicle === null; }
  get vehicle() { return this.#vehicle; }

  // చూడటం + రాయడం ఒకే చోట. మధ్యలో ఎవరూ దూరలేరు.
  tryClaim(v) {
    if (this.#vehicle !== null || this.size < v.size) return false;
    this.#vehicle = v;
    return true;
  }

  release() { const v = this.#vehicle; this.#vehicle = null; return v; }
}
```

```

class ParkingLot {
  constructor(spots) { this.spots = spots; }

  park(v) {
    for (const s of this.spots) if (s.tryClaim(v)) return { spotId: s.id, vehicle: v };
    return null;
  }
}

```

ఇక్కడ మారినది ఏమిటి:

- `findFreeSpot` మరియు `assign` పోయాయి. వాటి స్థానంలో ఒక `tryClaim`.
- `#vehicle` **private** అయింది. ఇది కీలకం – `public` గా ఉంటే ఎవరైనా `spot.vehicle = car` అని రాసి `tryClaim` ని పూర్తిగా దాటవేయగలరు. **నియమాన్ని దాటవేసే దారి ఉండకూడదు.**
- `park()` ఇప్పుడు కేవలం ఒక loop – "ప్రయత్నించు, విఫలమైతే తర్వాతిది".

అదే double-booking పరీక్షని మళ్ళీ నడుపుదాం:

```

Gate A → { spotId: 'M1', vehicle: { no: 'AP-09', size: 2 } }
Gate B → null
M1 lo: AP-09

```

Gate B కి `null` వచ్చింది – అంటే "ఈ spot దొరకలేదు, వేరేది చూడు". ఇదే సరైన ప్రవర్తన.

9. నిజమైన systems lo ఇది ఎలా ఉంటుంది

JavaScript single-threaded కాబట్టి మన `tryClaim` దానంతట అదే atomic. కానీ interviewer ఖచ్చితంగా అడుగుతాడు: "Java lo ఇది ఎలా?" – ఈ పట్టిక సిద్ధంగా ఉంచుకోండి.

సందర్భం	పరిష్కారం	ఖర్చు
ఒకే JVM, తక్కువ contention	<code>tryClaim</code> ని <code>synchronized</code> చేయడం	సులభం; కానీ ఒక్క spot కి ఒక్క lock అయితే మెరుగు
ఒకే JVM, ఎక్కువ contention	<code>AtomicReference</code> + <code>compareAndSet</code>	Lock-free, వేగం; కానీ రాయడం కష్టం
పలు servers, ఒకే DB	<code>UPDATE spots SET vehicle=? WHERE id=? AND vehicle IS NULL</code> – affected rows 1 అయితేనే గెలిచినట్టు	DB round trip
పలు servers, DB లేదు	Redis <code>SETNX</code> , లేదా ఒక distributed lock	ఒక కొత్త ఆధారపడటం

మూడో వరుసని గుర్తుపెట్టుకోండి

WHERE id=? AND vehicle IS NULL – ఆ రెండో షరత్ మొత్తం రహస్యం. అది లేకపోతే రెండు servers ఒకేసారి రాసేస్తాయి. అది ఉంటే, DB ఒక్కరినే గెలిపిస్తుంది, మరియు ఓడినవాడికి "0 rows updated" వస్తుంది.

ఇది **conditional update** అనే నమూనా, మరియు ఇది ప్రతిచోటా వస్తుంది – seat booking, inventory, wallet balance. ఇక్కడ నేర్చుకుంటే అక్కడ ఉచితం.

"అన్నిటికీ ఒకే lock పెడితే సరిపోదా?"

సరిపోతుంది – మరియు అది తప్పు కాదు, ముఖ్యంగా ఒక చిన్న lot కి. కానీ ఖర్చు తెలిసి ఉండాలి:

"మొత్తం lot కి ఒకే lock పెడితే సరళంగా ఉంటుంది, కానీ అప్పుడు ఒక్క సమయంలో ఒక్క gate మాత్రమే పనిచేస్తుంది – నాలుగు gates ఉన్నా. 500 spots ఉన్న lot lo peak hour lo అది ఒక queue అవుతుంది. నేను lock ని **spot స్థాయిలో** ఉంచుతాను, అప్పుడు వేర్వేరు spots కోసం gates సమాంతరంగా పనిచేస్తాయి."

ఇలాంటి జవాబు – రెండు ఎంపికలు, ఒక ఎంపిక, ఒక కారణం – ఇదే interviewer వెతుకుతున్నది.

Part 4 – మూడో విరుపు: పెరుగుతున్న pricing

ఇది ఒక నెమ్మదిగా వచ్చే విరుపు. ఒక్కసారిగా పగలదు – నాలుగు requirements తర్వాత మీరు గమనిస్తారు.

10. మూడో విరుపు – ఒక్కో నియమానికి ఒక్కో if

Interviewer అంటాడు: "ఇప్పుడు డబ్బు లెక్కించాలి. గంటకి ₹20." సులభం:

```
function fee1(hours) { return hours * 20; }
```

"Bike కి తక్కువ, truck కి ఎక్కువ ఉండాలి."

```
function fee2(type, hours) {  
  if (type === 'bike') return hours * 10;  
  if (type === 'car') return hours * 20;  
  return hours * 40;  
}
```

"Weekend lo 1.5 రెట్లు."

```
function fee3(type, hours, isWeekend) {  
  let rate = type === 'bike' ? 10 : type === 'car' ? 20 : 40;  
  if (isWeekend) rate *= 1.5;  
  return hours * rate;  
}
```

"మొదటి 30 నిమిషాలు ఉచితం. మరియు EV లకి 20% తగ్గింపు."

```
function fee4(type, hours, isWeekend, isEV) {  
  if (hours <= 0.5) return 0;  
  let rate = type === 'bike' ? 10 : type === 'car' ? 20 : 40;  
  if (isWeekend) rate *= 1.5;  
  let total = (hours - 0.5) * rate;  
  if (isEV) total *= 0.8;  
  return total;  
}
```

```

fee1(3) = 60
fee2(truck,3) = 120
fee3(truck,3,weekend) = 180
fee4(truck,3,weekend,EV) = 120

```

మూడో విరుపు - దీన్ని సంఖ్యలతో చూడండి

నాలుగు requirements. Parameters **1** → **4** కి పెరిగాయి. Function lo lines **1** → **7** కి పెరిగాయి. మరియు ప్రతి కొత్త నియమం రావడానికి - ఈ ఒక్క function ని తెరిచి మార్చాలి.

ఇప్పుడు ఊహించండి: "Corporate customers కి flat rate", "రాత్రి 10 తర్వాత సగం", "మొదటి visit ఉచితం", "monthly pass". వన్నెండో నియమం వచ్చేసరికి ఈ function ఎవరూ ముట్టుకోలేని స్థితికి చేరుతుంది - మరియు ప్రతి మార్పుతో పాత నియమాలు విరిగే ప్రమాదం ఉంటుంది.

ఇక్కడ అసలు తప్పు ఏమిటి?

ఆ fee4 function తప్పుగా రాయబడలేదు - అది సరైన జవాబే ఇస్తోంది. తప్పు ఏమిటంటే: ఇది మారే విషయాన్ని, మారని code lo కలిపి ఉంచింది.

Parking lot యొక్క నిర్మాణం (spots, tickets, allocation) దాదాపు ఎప్పుడూ మారదు. కానీ pricing - అది ప్రతి నెలా మారుతుంది, ఎందుకంటే అది ఒక business నిర్ణయం, ఒక సాంకేతిక నిర్ణయం కాదు.

నియమం: ఏ code ఎంత తరచుగా మారుతుందో దాన్ని బట్టి విడగొట్టు. తరచుగా మారేదాన్ని, అరుదుగా మారేదాని లోపల పెట్టకు.

11. Step - Pricing ని బయట పెట్టడం

మొదటి అడుగు: fee లెక్కింపుని ఒక వస్తువు గా మార్చడం. ParkingLot కి ఆ నియమం ఏమిటో తెలియనవసరం లేదు - దానికి కావలసింది ఒక్కటే: "ఈ ticket కి ఎంత?"

```

class HourlyRate {
    constructor(rates) { this.rates = rates; } // { SMALL: 10, MEDIUM: 20,
    LARGE: 40 }
    fee(ticket, hours) { return hours * this.rates[ticket.size]; }
}

```

అంతే. మొత్తం contract ఒక్క method: fee(ticket, hours).

ఇప్పుడు "weekend lo 1.5 రెట్లు" ఎలా చేర్చాలి? ఒక కొత్త class రామొచ్చు:

```

class WeekendHourlyRate {
    fee(ticket, hours) { /* ... */ }
}

```

కానీ ఇక్కడ ఒక సమస్య వస్తుంది. "Weekend" మరియు "మొదటి 30 నిమిషాలు ఉచితం" మరియు "EV discount" – ఇవి **కలిసి** రావోచ్చు. ప్రతి కలయికకీ ఒక class రాస్తే:

WeekendRate , FreeHalfHourRate , WeekendFreeHalfHourRate ,
WeekendFreeHalfHourEvRate ... – **మళ్ళీ అదే explosion**, ఈసారి classes రూపంలో.

12. Step – నియమాలని ఒకదాని మీద ఒకటి పేర్చడం

అసలు గమనించాల్సినది ఇది: ఈ నియమాలు **ఒకదాన్ని ఒకటి మారుస్తున్నాయి**, ఒకదాన్ని ఒకటి భర్తీ చేయట్లేదు.

- "మొదటి 30 నిమిషాలు ఉచితం" = ఉన్న లెక్క నుంచి అరగంట తీసేయడం
- "Weekend lo 1.5x" = ఉన్న లెక్క ని 1.5 తో గుణించడం
- "EV కి 20% తగ్గింపు" = ఉన్న లెక్క ని 0.8 తో గుణించడం

అంటే ప్రతి నియమం ఒక **ఛోర** – లోపలి లెక్కని చుట్టి, దాన్ని కొంచెం మార్చేది. ఇదే **Decorator pattern**.

```
class FreeFirstHalfHour {
  constructor(inner) { this.inner = inner; }
  fee(ticket, hours) {
    return hours <= 0.5 ? 0 : this.inner.fee(ticket, hours - 0.5);
  }
}

class WeekendSurcharge {
  constructor(inner, factor = 1.5) { Object.assign(this, { inner, factor }); }
  fee(ticket, hours) {
    return this.inner.fee(ticket, hours) * (ticket.isWeekend ? this.factor : 1);
  }
}

class EvDiscount {
  constructor(inner, off = 0.2) { Object.assign(this, { inner, off }); }
  fee(ticket, hours) {
    return this.inner.fee(ticket, hours) * (ticket.isEV ? 1 - this.off : 1);
  }
}
```

ప్రతి class **ఒక్క నియమం** మాత్రమే తెలుసు. మిగతావేమీ తెలియవు. ఇప్పుడు వాటిని పేర్చుదాం:

```
const pricing = new EvDiscount(
  new WeekendSurcharge(
    new FreeFirstHalfHour(
      new HourlyRate({ 1: 10, 2: 20, 3: 40 })))));
```


PRICING పొరలు · ఒక TRUCK, 3 గంటలు, WEEKEND, EV

బయటి పొర నుంచి లోపలికి, ఆపై లెక్క తిరిగి బయటికి

EvDiscount – "EV నా? అయితే $\times 0.8$ "

WeekendSurcharge – "Weekend నా? అయితే $\times 1.5$ "

FreeFirstHalfHour – "అరగంట లోపేనా? అయితే 0. లేకపోతే hours – 0.5"

HourlyRate – "గంటలు $\times$ ఆ size యొక్క rate"

లెక్క: $(3 - 0.5) \times 40 = 100 \rightarrow \times 1.5 = 150 \rightarrow \text{EV కాదు కాబట్టి} \times 1 = ₹150$

కొత్త నియమం కావాలా? ఒక కొత్త పొర రాసి, పేర్లులో చేర్చండి. ఉన్న ఒక్క class కూడా మారదు.

నిజంగా నడిపితే:

bike	3 గంటలు	→ ₹25
car	3 గం (EV)	→ ₹40
truck	3 గం (weekend)	→ ₹150
20 నిమిషాలు	మాత్రమే	→ ₹0

ప్రతి సంఖ్యనీ చేతితో తనిఖీ చేయొచ్చు: bike $(3-0.5) \times 10 = 25$ ✓ · car $(3-0.5) \times 20 \times 0.8 = 40$ ✓ · truck $(3-0.5) \times 40 \times 1.5 = 150$ ✓ · 20 నిమిషాలు = 0 ✓.

Strategy నా Decorator నా? ఇది మంచి interview ప్రశ్న, మరియు జవాబు: **రెండూ**. pricing అనే మొత్తం వస్తువు lot కి ఒక **Strategy** – దాన్ని పూర్తిగా వేరేదానితో మార్చొచ్చు. ఆ strategy లోపల నియమాలు **Decorator** లుగా పేర్చబడ్డాయి. Pattern పేరు కంటే ముఖ్యమైనది: **lot కి pricing నియమం తెలియదు, మరియు ఒక్కో నియమానికి ఒక్కో class ఉంది.**

Part 5 – Spot ఎంపిక, floors

13. Step – "ఏ spot ఇవ్వాలి" కూడా ఒక మారే నియమమే

ఇప్పటివరకు `park()` మొదటి సరిపోయే spot ని ఇస్తోంది. కానీ ఆలోచించండి – lot lo ఒక SMALL spot, ఒక LARGE spot ఖాళీగా ఉన్నాయి. ఒక **bike** వచ్చింది. మొదటిది SMALL అయితే సరే. కానీ spots క్రమం వేరుగా ఉండి LARGE ముందు వస్తే – **bike ఒక truck spot ని ఆక్రమిస్తుంది**, ఆపై truck వస్తే చోటు ఉండదు.

కాబట్టి "ఏ spot ఇవ్వాలి" అనేది కూడా ఒక నియమం, మరియు అది కూడా మారుతుంది:

- **NearestFirst** – మొదటిది ఇవ్వ. వేగం; user కి తక్కువ నడక.
- **BestFit** – సరిపోయే వాటిలో **అతి చిన్నది** ఇవ్వ. పెద్ద spots ని కాపాడుతుంది.
- రాత్రిపూట – ఒకే floor lo ఉంచి మిగతా floors lo lights ఆర్పొచ్చు.

మారే నియమం = బయట పెట్టాలి. ఇప్పుడు ఇది మీకు అలవాటైన కదలిక:

```
class NearestFirst {
  choose(spots, v) { return spots.find((s) => s.tryClaim(v)) ?? null; }
}

class BestFit {
  choose(spots, v) {
    const fits = spots.filter((s) => s.isFree && s.size >= v.size)
                        .sort((a, b) => a.size - b.size);      // చిన్నవి ముందు
    for (const s of fits) if (s.tryClaim(v)) return s;         // claim ఇంకా atomic
    return null;
  }
}
```

BESTFIT LO ఒక సూక్ష్మమైన, కానీ ముఖ్యమైన వివరం

`filter` తో ఖాళీ spots జాబితా తీశాం, ఆపై `tryClaim` చేశాం. ఆ రెండింటి మధ్య మళ్ళీ ఒక ఖాళీ ఉంది – **\$7 lo చూసిన అదే check-then-act!**

కానీ ఈసారి అది ప్రమాదకరం కాదు, ఎందుకంటే **నిజమైన నిర్ణయం ఇంకా tryClaim లోపలే** జరుగుతోంది. ఈ `filter` కేవలం ఒక ప్రాధాన్యత క్రమం. ఎవరైనా మధ్యలో ఒక spot తీసుకుంటే, `tryClaim false` ఇస్తుంది, మరియు మనం తర్వాతిదానికి వెళ్తాం – అందుకే అక్కడ `for loop` ఉంది, `fits[0]` కాదు.

Interview lo ఈ వివరాన్ని బయటికి చెప్పండి. "నేను `filter` చేస్తున్నాను కానీ నిర్ణయం ఇంకా atomic claim దగ్గరే ఉంది" – ఇది మీరు concurrency ని ఒకసారి నేర్చుకుని మర్చిపోలేదని చూపిస్తుంది.

14. Floors, మరియు Floor ని class చేయాలా వద్దా

Requirement lo "floors" అనే పదం ఉంది. కాబట్టి Floor ఒక class కావాలా?

§5 lo చెప్పిన నియమాన్ని వాడండి: **తేడా data lo ఉంటే field; తేడా ప్రవర్తనలో ఉంటే class.**

ఒక floor ఏమి చేస్తుంది? ఇప్పటివరకు – ఏమీ లేదు. అది కేవలం spots ని గుంపుగా చేస్తుంది. కాబట్టి Spot కి ఒక floor field చాలు:

```
new Spot('F1-M1', Size.MEDIUM, 1) // id, size, floor
```

Interview lo ఇలా చెప్పండి:

"I'm not making Floor a class right now. A floor has no behaviour here – it just groups spots, and a field on Spot captures that. But I'd change my mind the moment a floor gets its own rules: a floor that's reserved for staff, a floor with a separate entry gate, or per-floor occupancy counters for the display board at the entrance. At that point Floor owns something, and it becomes a class.

The same reasoning applies to Gate – right now a gate is just a caller, so I don't model it. If gates needed their own state, like being open or closed for maintenance, that would change."

ఇదే జవాబు Floor, Gate, Display, Attendant – అన్నిటికీ వర్తిస్తుంది. **Requirement lo ఒక noun ఉంది కదా అని class రాయకండి.** ఆ noun కి బాధ్యత ఉందా అని అడగండి.

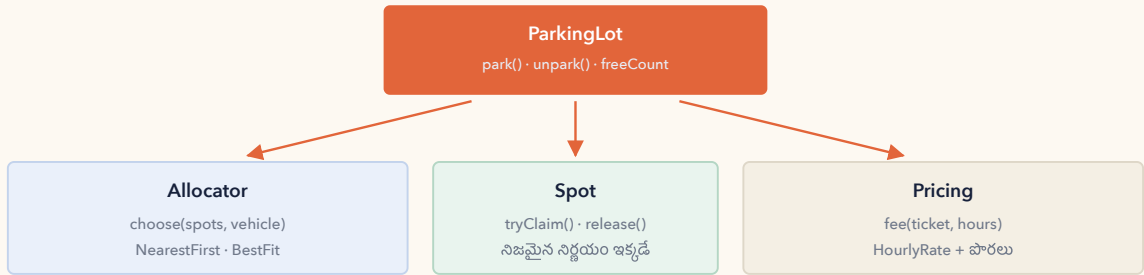
Part 6 – పూర్తి system

ఐదు చిన్న classes, రెండు pluggable కుటుంబాలు. ప్రతి పంక్తికీ ఒక కారణం ఉంది, మరియు ఆ కారణం మీకు తెలుసు.

15. మొత్తం code ఒకే చోట

నిర్మాణం · ఎవరికి ఏమి తెలుసు, ఎవరికి తెలియదు

బాణం అర్థం: "దీని గురించి తెలుసు". తెలియకపోవడమే ఇక్కడ design.



ఇక్కడ ముఖ్యమైనది – ఎవరికి ఏమి తెలియదు

ParkingLot కి pricing నియమం తెలియదు · allocation నియమం తెలియదు
Spot కి lot గురించి, ఇతర spots గురించి తెలియదు – అది తన స్థితిని మాత్రమే కాపాడుతుంది
Pricing కి spots, floors, gates గురించి తెలియదు – అది ticket మరియు hours మాత్రమే చూస్తుంది
ఈ "తెలియకపోవడం" వల్లనే నాలుగింటినీ విడిగా మార్చొచ్చు, విడిగా test చేయొచ్చు

```
const Size = Object.freeze({ SMALL: 1, MEDIUM: 2, LARGE: 3 });

class Vehicle {
  constructor(plate, size, { isEV = false } = {}) {
    Object.assign(this, { plate, size, isEV });
    Object.freeze(this); // vehicle వచ్చాక దాని గుర్తింపు మారదు
  }
}

class Spot {
  #vehicle = null;
  constructor(id, size, floor) { Object.assign(this, { id, size, floor }); }
  get isFree() { return this.#vehicle === null; }
  get vehicle() { return this.#vehicle; }
  tryClaim(v) { // చూడటం + రాయడం, విడదీయలేనివి
    if (this.#vehicle !== null || this.size < v.size) return false;
    this.#vehicle = v;
    return true;
  }
  release() { const v = this.#vehicle; this.#vehicle = null; return v; }
}

// ---- మారే భాగం 1: ఏ spot ఇవ్వాలి ----
class NearestFirst {
```

```

    choose(spots, v) { return spots.find((s) => s.tryClaim(v)) ?? null; }
}
class BestFit {                                // అతి చిన్న సరిపోయే spot
    choose(spots, v) {
        const fits = spots.filter((s) => s.isFree && s.size >= v.size)
            .sort((a, b) => a.size - b.size);
        for (const s of fits) if (s.tryClaim(v)) return s;
        return null;
    }
}

// ---- మూరే భాగం 2: ఎంత డబ్బు ----
class HourlyRate {
    constructor(rates) { this.rates = rates; }
    fee(ticket, hours) { return hours * this.rates[ticket.size]; }
}
class FreeFirstHalfHour {
    constructor(inner) { this.inner = inner; }
    fee(ticket, hours) { return hours <= 0.5 ? 0 : this.inner.fee(ticket, hours - 0.5); }
}
class WeekendSurcharge {
    constructor(inner, factor = 1.5) { Object.assign(this, { inner, factor }); }
    fee(ticket, hours) {
        return this.inner.fee(ticket, hours) * (ticket.isWeekend ? this.factor : 1);
    }
}
class EvDiscount {
    constructor(inner, off = 0.2) { Object.assign(this, { inner, off }); }
    fee(ticket, hours) { return this.inner.fee(ticket, hours) * (ticket.isEV ? 1 - this.off : 1); }
}

class Ticket {
    constructor(spot, vehicle, inTime) {
        Object.assign(this, { spotId: spot.id, floor: spot.floor, size: vehicle.size,
            plate: vehicle.plate, isEV: vehicle.isEV, inTime });
        this.isWeekend = false;
        Object.seal(this);                // కొత్త fields చేర్చలేరు
    }
}

class ParkingLot {
    #tickets = new Map();
    constructor(spots, { allocator = new NearestFirst(), pricing } = {}) {
        Object.assign(this, { spots, allocator, pricing });
    }
    get freeCount() { return this.spots.filter((s) => s.isFree).length; }

    park(vehicle, inTime = 0) {
        const spot = this.allocator.choose(this.spots, vehicle);
        if (!spot) return null;           // వోటు లేదు
        const t = new Ticket(spot, vehicle, inTime);
        this.#tickets.set(vehicle.plate, t);
        return t;
    }

    unpark(plate, outTime) {
        const t = this.#tickets.get(plate);

```

```

    if (!t) throw new Error(`NO_TICKET: ${plate}`);
    this.spots.find((s) => s.id === t.spotId).release();
    this.#tickets.delete(plate);
    return this.pricing.fee(t, outTime - t.inTime);
  }
}

```

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§4	<code>spots</code> , <code>park</code> , <code>unpark</code>	మౌలిక అస్థిపంజరం
§5 (ఎరుపు)	<code>Size</code> , <code>Spot.size</code>	అన్ని చోట్లా ఒకేలా కావు
§7 (ఎరుపు)	<code>tryClaim</code> , <code>#vehicle</code> private	Check-then-act race
§10 (ఎరుపు)	<code>pricing</code> బయటికి	ప్రతి కొత్త నియమం code ని తెరిపించేది
§12	Decorator పాఠాలు	నియమాలు కలిపి వస్తాయి
§13	<code>allocator</code>	"ఏ spot" కూడా ఒక మారే నియమమే
ఇక్కడ	<code>Ticket</code> , <code>#tickets</code> Map	Bill కి inTime, మరియు plate నుంచి వెతకడం

16. నడిపి చూద్దాం – ticket నుంచి బిల్లు వరకు

```

const spots = [new Spot('F1-S1', Size.SMALL, 1), new Spot('F1-M1', Size.MEDIUM, 1),
  new Spot('F1-L1', Size.LARGE, 1), new Spot('F2-M1', Size.MEDIUM, 2)];

const pricing = new EvDiscount(new WeekendSurcharge(
  new FreeFirstHalfHour(new HourlyRate({ 1: 10, 2: 20, 3: 40 } ))));

const lot = new ParkingLot(spots, { allocator: new BestFit(), pricing });

const bike = new Vehicle('TS-01', Size.SMALL);
const car = new Vehicle('TS-02', Size.MEDIUM, { isEV: true });
const truck = new Vehicle('TS-03', Size.LARGE);

const t1 = lot.park(bike, 0), t2 = lot.park(car, 0), t3 = lot.park(truck, 0);
console.log('bike →', t1.spotId, '| car →', t2.spotId, '| truck →', t3.spotId);
console.log('ఖాళీ spots:', lot.freeCount);
console.log('ఇంకే truck →', lot.park(new Vehicle('TS-04', Size.LARGE), 0));

console.log('bike 3 గంటలు → ₹' + lot.unpark('TS-01', 3));
console.log('car 3 గం (EV) → ₹' + lot.unpark('TS-02', 3));
t3.isWeekend = true;
console.log('truck 3 గం (weekend) → ₹' + lot.unpark('TS-03', 3));
const quick = new Vehicle('TS-06', Size.SMALL);
lot.park(quick, 0);
console.log('20 నిమిషాలు మాత్రమే → ₹' + lot.unpark('TS-06', 0.33));

```

```
bike → F1-S1 | car → F1-M1 | truck → F1-L1
ఖాళీ spots: 1
ఇంకో truck → null
bike 3 గంటలు → ₹25
car 3 గం (EV) → ₹40
truck 3 గం (weekend) → ₹150
20 నిమిషాలు మాత్రమే → ₹0
```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

bike → F1-S1 . BestFit పనిచేసింది. Bike కి SMALL, MEDIUM, LARGE – మూడూ సరిపోతాయి, కానీ అది **అతి చిన్నదాన్ని** ఎంచుకుంది. NearestFirst అయితే కూడా ఇదే వచ్చేది (S1 మొదటిది) – కానీ spots క్రమం వేరుగా ఉంటే తేడా కనిపించేది.

ఖాళీ spots: 1 . మూడు నిండాలు, F2-M1 మిగిలింది.

ఇంకో truck → null . మిగిలినది MEDIUM, truck కి LARGE కావాలి. సరిగ్గా తిరస్కరించింది. ఇది **"lot నిండింది" కాదు – "ఈ vehicle కి చోటు లేదు". తేడా ముఖ్యం.**

బిల్లులు. నాలుగు పొరలూ పనిచేస్తున్నాయి: bike $(3-0.5) \times 10 = 25$; car $(3-0.5) \times 20 \times 0.8 = 40$ (EV తగ్గింపు); truck $(3-0.5) \times 40 \times 1.5 = 150$ (weekend); 20 నిమిషాలు = 0 (ఉచిత అరగంట).

ఈ output ని మరే నడిపి చూడండి. అపై ఒక ప్రయోగం చేయండి – BestFit ని NearestFirst కి మార్చి, spots క్రమాన్ని తిప్పి చూడండి. Bike ఒక LARGE spot ని ఆక్రమించడం మరే చూస్తారు. §13 lo చెప్పిన సమస్య అప్పుడు కంటికి కనిపిస్తుంది.

Part 7 – Interview Io

17. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

నిపుణి ముఖ్యం – మొదటి 12 నిమిషాలు code లేకుండా

6 నిమి	Clarify – "ఎన్ని gates?" ని మర్చిపోవద్దు (§3)
6 నిమి	Nouns → classes · Floor/Gate ని ఎందుకు వదిలేస్తున్నారో చెప్పండి (§14)
14 నిమి – Spot, Vehicle, Lot	ఇక్కడ tryClaim ని మరే లేవనెత్తండి
8 నిమి	Pricing – Decorator పేర్లు
6 నిమి	Allocator Strategy
5 నిమి	Follow-ups, extensibility

ఏమి తప్పక చెప్పాలి

- "ఎన్ని gates?" – మరే అడగాలి (§3). ఇది concurrency తలుపు తెరుస్తుంది.
- Check-then-act race – tryClaim ని ఎందుకు ఒక method చేశారో (§7-8). ఇదే ఈ interview యొక్క అసలు ప్రశ్న.
- Pricing ఎందుకు బయట (§10) – "ఇది business నియమం, అది ప్రతి నెలా మారుతుంది".
- Floor ని class చేయకపోవడం (§14) – YAGNI ని ఒక కారణంతో చెప్పడం.

ఏమి వదిలేయాలి

- నాలుగు pricing decorators రాయొద్దు – రెండు రాసి, "మిగతావి ఇదే నమూనా" అని చెప్పండి.
- BestFit రాయొద్దు – NearestFirst రాసి, "allocator ఒక interface, best-fit ఒక కొత్త class" అని చెప్తే చాలు.
- toString, freeCount – ఇవి debugging కోసం, design కోసం కాదు.

18. నీటితో చెప్పాల్సిన English script

"Let me start with the questions that would change my design. How many entry gates – one or several? Can a smaller vehicle take a larger spot? And is pricing a flat hourly rate, or does it vary?

I'll assume four gates, smaller-fits-in-larger, and variable pricing, since those are the interesting versions.

The core objects are Vehicle, Spot, Ticket and ParkingLot. I'm not making Floor or Gate classes – a floor has no behaviour here, it just groups spots, so it's a field on Spot. I'd revisit that if a floor got its own rules, like staff-only access.

Now the part I want to get right. The naive park() is find a free spot, then assign it. That's a **check-then-act race** – with four gates, two can both see the same last spot as free and both issue a ticket for it. So I'm putting the claim on the Spot itself: tryClaim(vehicle) checks and writes in one operation and returns a boolean, and the vehicle field is private so nobody can bypass it. In Java I'd make that synchronized per spot rather than one lock over the whole lot, so gates don't serialise. If this were multiple servers against a database, it becomes a conditional update – WHERE id = ? AND vehicle IS NULL – and you check the affected row count.

For pricing, I won't put the rules inside the lot. Pricing is a business rule that changes far more often than the structure does. So the lot holds a pricing object with a single fee(ticket, hours) method, and the individual rules – free first half hour, weekend surcharge, EV discount – are decorators wrapping each other. A new rule is a new class, not an edit.

Same reasoning for which spot to hand out: nearest-first is fine, but best-fit stops a motorcycle from occupying a truck bay, so that's a strategy too.

One weakness in what I've written: my unpark does a linear scan to find the spot from the ticket. That's O(n) and I'd keep a spot-id map instead."

ఆ చివరి paragraph – సొంత లోపాన్ని మీరే చూపించడం. దీన్ని ఎప్పుడూ వదలొద్దు.

19. Follow-ups – మరియు మన design ఎలా తట్టుకుంటుంది

Follow-up	మన design ఏం చేస్తుంది	మారే classes
"EV charging spots కావాలి"	Spot కి hasCharger ; BestFit lo ఒక filter	Spot + allocator
"Handicapped spots"	అదే నమూనా – ఒక flag + allocation నియమం	0 కొత్త concepts

Follow-up	మన design ఏం చేస్తుంది	మారే classes
"Monthly pass holders కి ఉచితం"	కొత్త <code>PassHolderFree</code> decorator	+1 కొత్తది
"రాత్రి 10 తర్వాత సగం rate"	కొత్త <code>NightRate</code> decorator	+1 కొత్తది
"Entrance lo 'ఖాళీ: 42' board"	<code>freeCount</code> ఇప్పటికే ఉంది; floor వారీగా కావాలంటే ఒక group-by	0
"Reservation – ముందే book చేసుకోవడం"	<code>Spot</code> కి ఒక కొత్త state (<code>RESERVED</code>); <code>tryClaim</code> lo ఒక షరతు	Spot
"పలు servers"	<code>tryClaim</code> → DB conditional update (§9)	Spot యొక్క అమలు
"Lot నిండిందని గేటు దగ్గరే అపాలి"	<code>park()</code> కి ముందు <code>freeCount</code> – కానీ అది మళ్ళీ check-then-act!	కింద చూడండి

ఆ చివరి దానికి జాగ్రత్తగా జవాబు

"That one's a trap, and it's the same trap as before. If I check `freeCount > 0` at the gate and then let the car in, the last spot can be taken while the car is driving to it. So the display is **advisory, not a guarantee** – it can say 'Full' to stop new cars, but the actual reservation still has to happen at claim time. I'd keep the gate check as an optimisation to avoid letting cars in pointlessly, and keep `tryClaim` as the single source of truth."

ఈ జవాబు చాలా బలమైనది, ఎందుకంటే ఇది మీరు §7 lo నేర్చుకున్నదాన్ని ఒక కొత్త చీట గుర్తించారని చూపిస్తుంది. చాలా మంది అక్కడ మళ్ళీ అదే తప్పు చేస్తారు.

20. ఏమి నేర్చుకున్నాం

Parking lot గురించి నేర్చుకున్నది కొంచెమే. నిజంగా నేర్చుకున్నవి ఇవి:

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
తేడా data lo ఉంటే field, ప్రవర్తనలో ఉంటే class	Vehicle రకాలు, Floor (§5, §14)	ప్రతి modelling నిర్ణయంలో
Check-then-act race	రెండు gates, ఒకే spot (§7)	Seat booking, inventory, wallet, LRU cache
Conditional update	<code>WHERE vehicle IS NULL</code> (§9)	ఏ distributed claim lo అయినా

అల్ చన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
మారే business నియమాన్ని బయట పెట్టడం	Pricing (§11)	Discounts, fees, retry policy, rate limits
నియమాలు కలిపి వస్తే Decorator	4 pricing పొరలు (§12)	Logging, auth, caching wrappers
సలహా vs హామీ	Display board vs claim (§19)	ఏ cache, ఏ counter lo అయినా

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo తర్వాతివి: **02 LRU & LFU Cache** (అక్కడ మళ్ళీ $O(1)$ claim, మరియు eviction ఒక Strategy), **03 Rate Limiter**, **04 BookMyShow** (అక్కడ seat locking సరిగ్గా ఈ §7 యొక్క పెద్ద రూపం).

వేగవంతమైన revision కోసం – [LLD_Design_Problems_Telugu.pdf](#) lo Problem 01. సాధారణ సూత్రాల కోసం – [LLD_Telugu.pdf](#) §29 (Strategy), §18 (Decorator), §36 (Concurrency).

Parking Lot – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output నిజంగా `node` lo run చేసి తీసినదే ✓